Reporte de Laboratorio: Resiliencia en catalog-service

Laboratorios 1.1 y 1.2 · Unidad 1: Arquitectura de alta disponibilidad

Alumno: Jose Maria Romero Davila

Maestra: Ivonne Tatiana Alcántara Llanas

1. Introducción y objetivo

En esta práctica se evaluó el comportamiento del catalog-service (proyecto-base-unidad-01, Quarkus) ante fallas simuladas de base de datos: 30 % de errores HTTP 500 y 20 % de latencia de ~3000 ms. El objetivo fue medir una línea base sin resiliencia y comparar el efecto de agregar, de forma incremental, los patrones Timeout, Retry, Fallback y CircuitBreaker de MicroProfile Fault Tolerance.

2. Metodología

Se ejecutó el servicio en modo desarrollo (./mvnw quarkus:dev) sobre Ubuntu 26.04.1 LTS, OpenJDK 25 y Maven 3.9.12. En cada etapa se enviaron 200 solicitudes secuenciales a GET /v1/products con el script test-availability.sh (curl + bash), que registra código HTTP, time_total y cuerpo de la respuesta. La disponibilidad se calculó como respuestas HTTP 200 entre solicitudes totales.

Etapa	Configuración en getProducts()	Evidencia
Línea base	Sin patrones de resiliencia	availability-baseline.txt
Etapa 1	@Timeout(800) + @Retry(maxRetries = 2, delay = 150)	availability-modification1.txt
Etapa 2	Etapa 1 + @Fallback(fallbackMethod = "getCatalogFallback")	availability-modification2.txt
Etapa 3	Etapa 2 + @CircuitBreaker(requestVolumeThreshold = 10, failureRatio = 0.7, delay = 5000)	availability-modification3.txt

3. Resultados

Métrica	Línea base	Etapa 1 Timeout + Retry	Etapa 2 + Fallback	Etapa 3 + CircuitBreaker
SUCCESS	131 (65.5 %)	87 (43.5 %)	77 (38.5 %)	20 (10.0 %)
DEGRADED_CACHE	0	0	123 (61.5 %)	180 (90.0 %)
Errores HTTP 500	69 (34.5 %)	113 (56.5 %)	0	0
Disponibilidad	0.6550	0.4350	1.0000	1.0000
Latencia máxima	~3.01 s	~1.97 s	~1.45 s	~1.25 s
Latencia media degradada	N/A	N/A	~1.1 s	~0.35 s

Tabla 1. Comparación de las cuatro mediciones (200 solicitudes cada una).

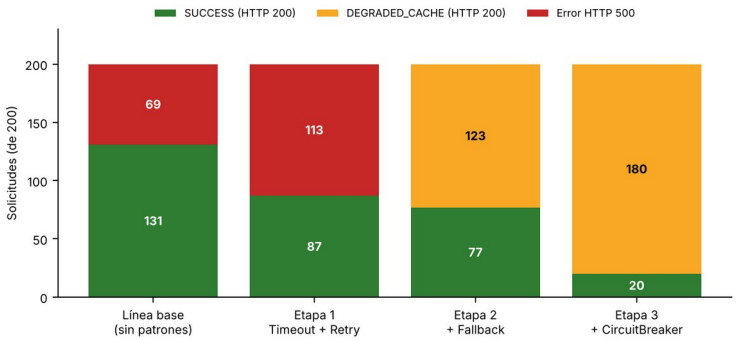


Figura 1. Composición de las respuestas por etapa.

4. Análisis

- **Línea base:** la disponibilidad fue de 65.50 %, con 34.5 % de errores 500 y picos de ~3 s por dependencia externa. El problema no es de cómputo, sino de bloqueo por la base de datos.
- **Timeout + Retry:** el @Timeout(800) eliminó los bloqueos de 3 s (máximo ~1.97 s), pero la disponibilidad bajó a 43.50 %. Esta caída no es una regresión: en esta etapa el escenario de fallos es más agresivo (60 % de solicitudes fallan en los 3 intentos), por lo que no es directamente comparable con la línea base.
- **Fallback:** los 113 errores 500 pasaron a respuestas HTTP 200 con DEGRADED_CACHE; la disponibilidad llegó a 100 %. El costo es que 61.5 % de las respuestas son datos en caché, es decir, se prioriza disponibilidad sobre consistencia.
- **CircuitBreaker:** mantuvo 100 % de disponibilidad y redujo la latencia media de las respuestas degradadas de ~1.1 s a ~0.35 s, porque con el circuito abierto se evitan reintentos innecesarios. Los SUCCESS bajaron de 77 a 20 porque, mientras el circuito está abierto, las solicitudes ya no llegan a la base de datos primaria.

5. Conclusiones

- Sin patrones de resiliencia, el servicio solo atendió correctamente dos de cada tres solicitudes y sufrió bloqueos de ~3 s.
- Timeout acota la latencia, Retry ofrece nuevos intentos y Fallback es el patrón que garantiza una respuesta al cliente (0 errores HTTP en las etapas 2 y 3).
- CircuitBreaker protege al backend y acelera las respuestas degradadas; la combinación completa deja al sistema con 100 % de disponibilidad y latencia máxima de ~1.25 s, a cambio de servir datos en caché la mayor parte del tiempo.

6. Referencias

Eclipse Foundation. (2023). *MicroProfile Fault Tolerance specification, 4.0.* https://download.eclipse.org/microprofile/microprofile-fault-tolerance-4.0/microprofile-fault-tolerance-spec-4.0.html

Quarkus. (s. f.). *SmallRye Fault Tolerance [Guía].* https://quarkus.io/guides/smallrye-fault-tolerance

Microsoft. (s. f.). *Circuit Breaker pattern. Azure Architecture Center.* https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker